



UpDown: A Supercomputer Co-designed for Scalable Graph Processing

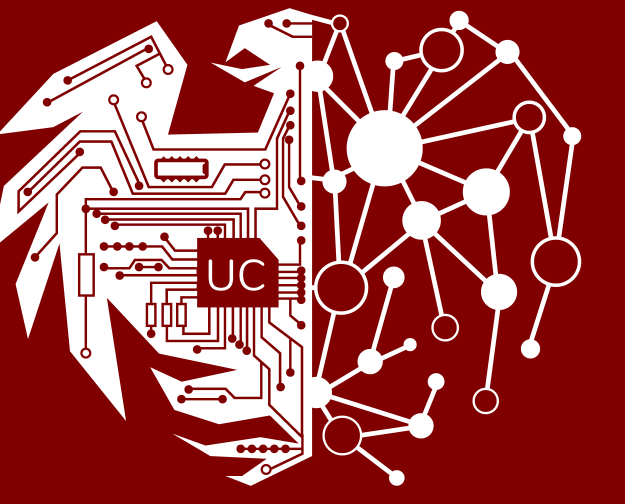
Andrew A. Chien^{1 2} Charles Colley³, Jianru Ding¹, Alexander Fell¹, David F. Gleich³, Henry Hoffmann¹, Moubarak Jeje⁴, Rajat Khandelwal¹, Yanjing Li¹, Jose M Monsalve-Diaz², Marziyeh Nourian¹, Andronicus Rajasukumar¹, Jiya Su¹, Tianshuo Su¹, Yuqing (Ivy) Wang¹, Wenyi Wang¹, Ruiqi Xu¹, Tianchi Zhang¹

¹University of Chicago

²Argonne National Laboratory

³Purdue University

⁴Tactical Computing Labs



Why Graph Processing Is Hard

Traditional supercomputers have focused on dense computation performance as exemplified by HPL. Graph processing applications differ with extreme irregularity (10^9 imbalance in skewed, real-world graphs) that produces unpredictable work, parallelism, memory access, and communication.

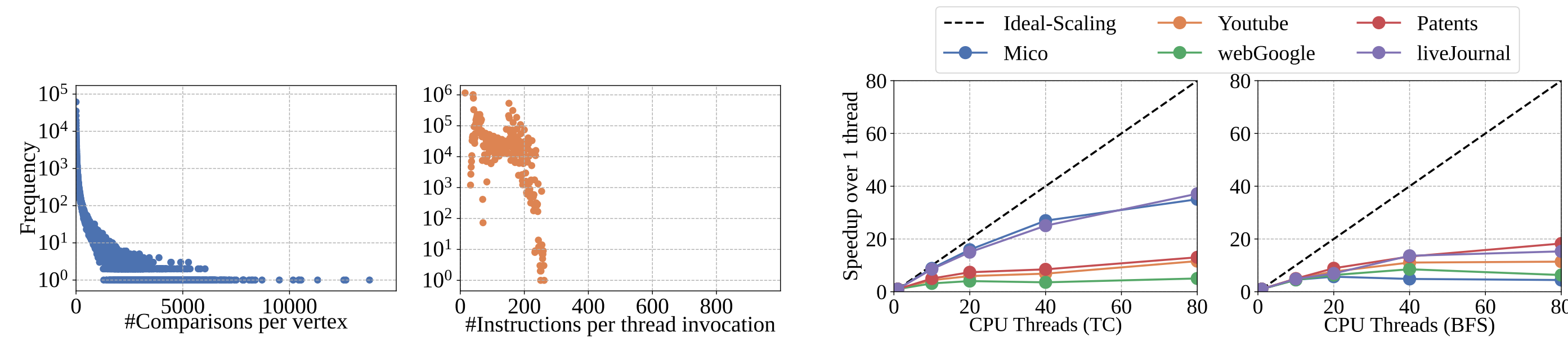


Figure 1. Graph computing has per-vertex parallelism with skew of 13.7k-fold (left TC on Yo dataset); Speedup vs. Threads on Intel Xeon on Triangle Counting and BFS (right).

- Irregular applications on skewed graphs exhibit poor data reuse, as well as irregular computation structure, memory reference, and parallelism.
- Today's scalable parallel machines excel at dense regular computations by exploiting structured work and scalable data reuse. Larger grain sizes enable amortization of high scheduling, communication, and synchronization costs, but sacrifice 1000-fold parallelism and limit scalability.
- Results show the best software frameworks, after trading away massive-parallelism, deliver poor scaling — 50% efficiency (Triangle Counting) and even worse 25% efficiency (BFS) at 80 threads with hyperthreading. For irregular computations with low data reuse, caches do not work well and can even inhibit performance.

UpDown System Overview

The UpDown system is a scalable parallel computer, consisting of many nodes connected by a scalable direct network.

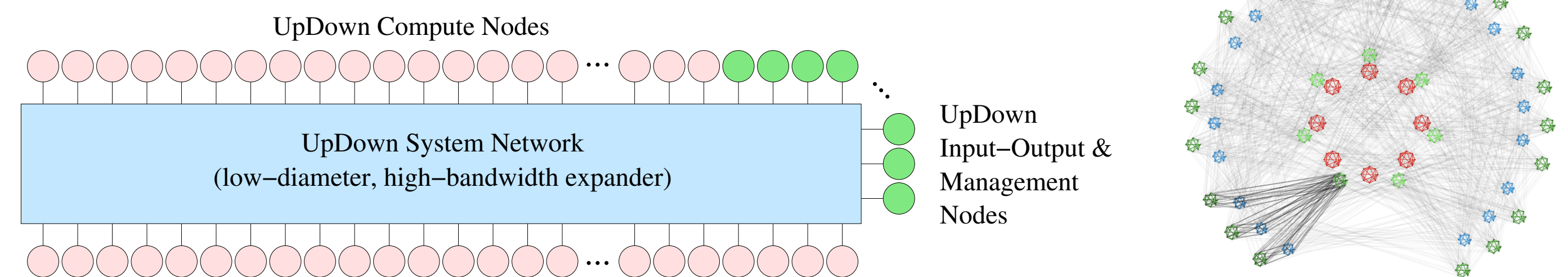


Figure 2. The UpDown System consists of compute, input/output, and management nodes (left). The UpDown system network is a scalable Polarstar network (right).

- Compute, input/output, and management nodes are all connected by the high-performance system network.
- The system network is a low-latency direct network, provisioned for high bandwidth to meet target applications that exhibit low data reuse.
- The network employs an expander graph-based topology, Polarstar, and high-speed links to create scalable global bandwidth; the diameter-3 topology provides a nearly flat network locality structure.

Node Architecture

The key element of an UpDown node is a set of 32 UpDown accelerators, each consisting of 64 lanes, 2,048 lanes in total.

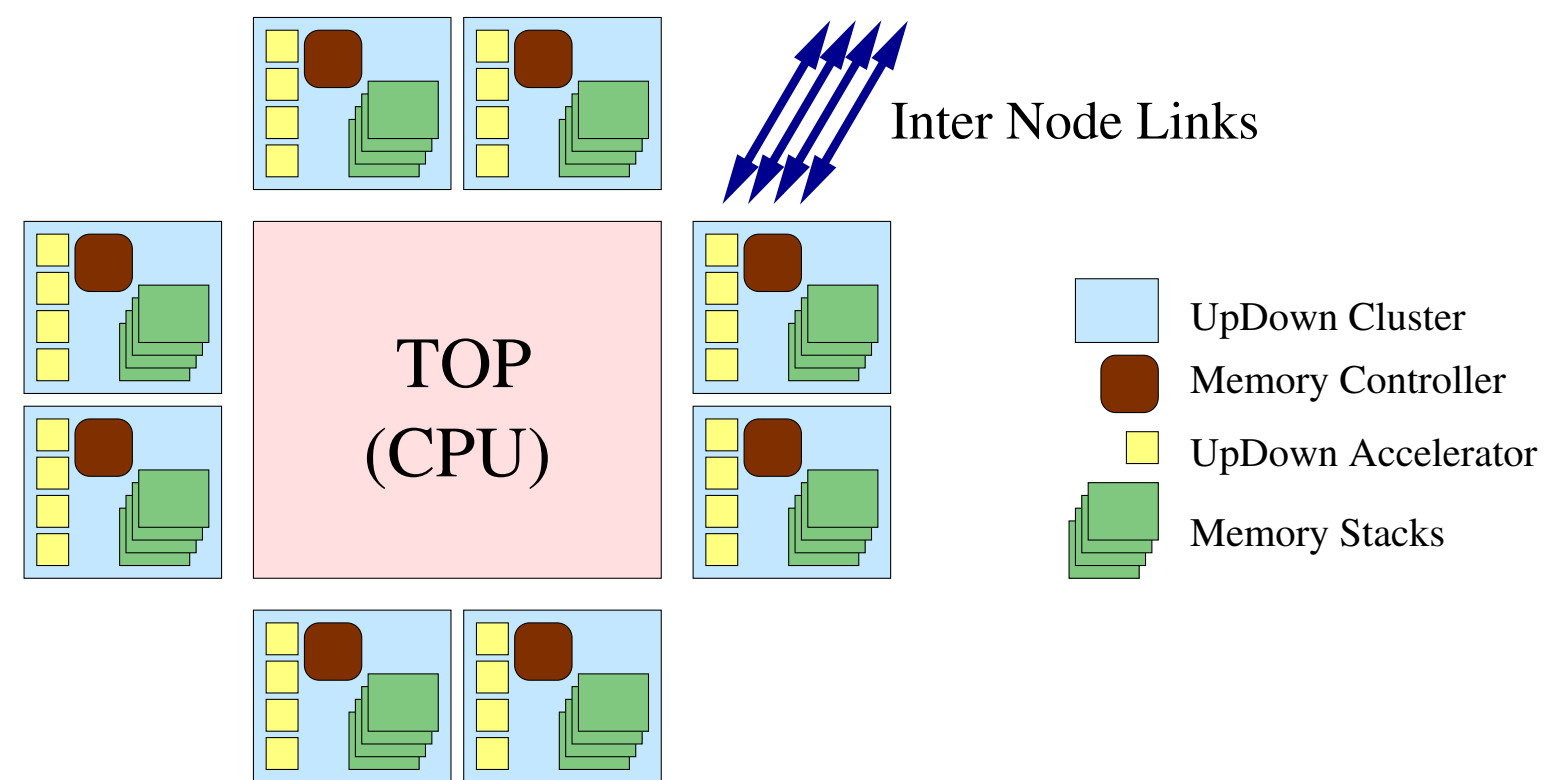


Figure 3. Each Updown node includes a multicore TOP CPU, a set of UpDown accelerators, HBM DRAM memory stacks, and system network connections.

- A multicore CPU with cache hierarchy is used for computations with high data reuse; coherence is only within the node.
- The node also includes 8 HBM stacks (HBM3e in current design) and connection to the high-speed system network.
- The UpDown accelerators are the key architectural innovation. With no caches, they provide high-performance on low-reuse computations; between scratchpad and DRAM, data movement and consistency are software managed.

Lane Microarchitecture and Key Mechanisms

Each UpDown accelerator consists of 64 UpDown lanes, designed for efficient fine-grained parallelism. Each lane executes independently and includes mechanisms for efficient response to hardware asynchrony and variable latency.

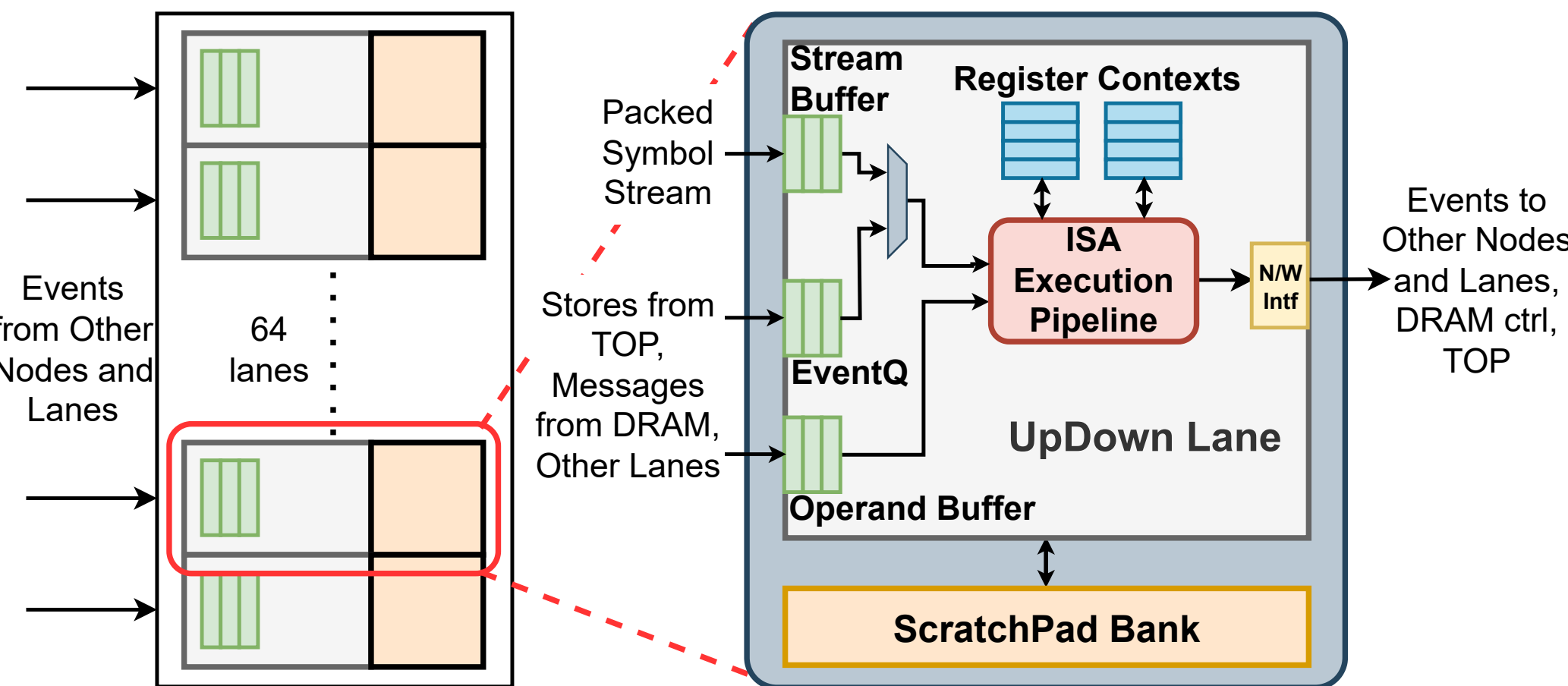


Figure 4. Fine-grained parallelism is supported by UpDown Lanes, with 1-cycle, event-driven scheduling, 128-way multithreading, and ISA split-transaction DRAM access. Thread invocations with as low as 10 instructions can be efficient.

- Low Latency Events:** Hardware event queues and hardware event-driven thread scheduling enable 1-cycle latency.
- Effective Short Thread Activations:** Direct access to event operands, hardware-supported thread create, schedule, terminate, and direct ISA messaging for 1–8 words.
- Direct ISA split-transaction DRAM access:** One to eight-word load/store operations to DRAM; asynchronous return enables flexible, software memory-latency tolerance.

Flexible Event-Driven Software

UpDown events have varied use in our graph and sparse matrix applications. We use TC as a representative example.

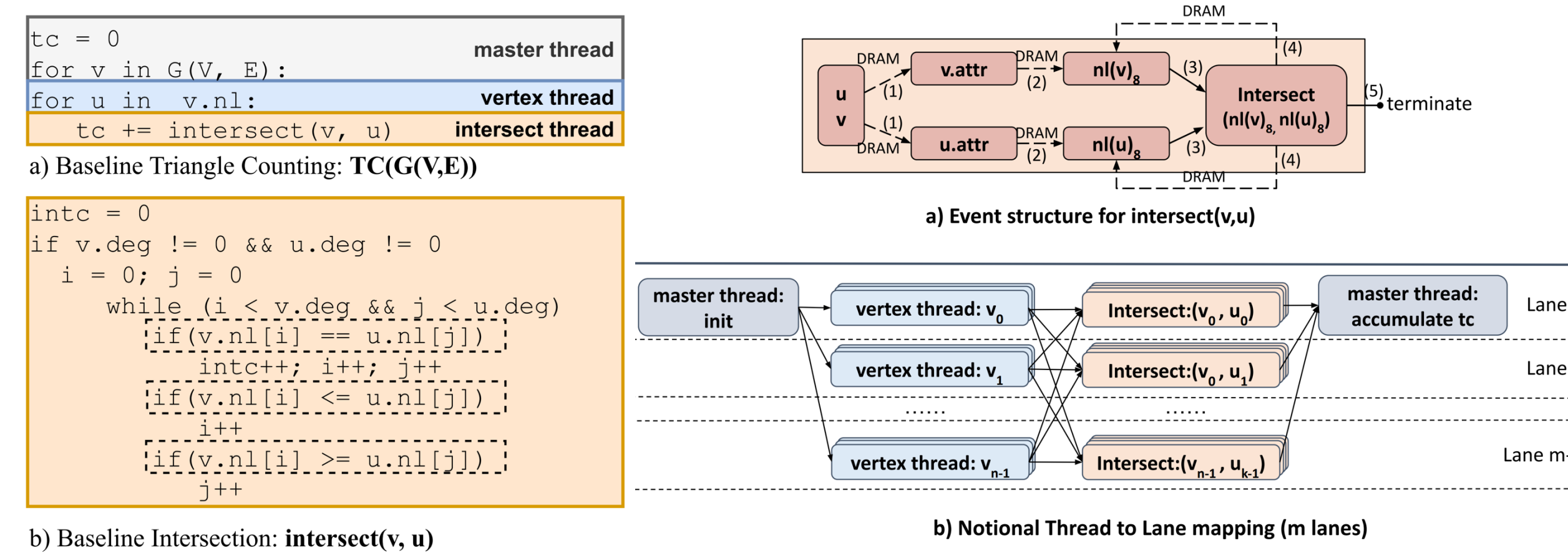


Figure 5. Triangle Counting baseline (left), UpDown implementation (right).

- UpDown's fine-grained parallelism allows all vertex iterations and intersection iterations to be executed in parallel.
 - UpDown currently supports a wide range of graph and sparse matrix applications, including PageRank, BFS, SpMV, Triangle Counting, Diamond Counting, Four-Cycle Counting, K-Truss, K-Core, Connected Components, Strongly Connected Components, Partial Match, and Louvain Community Detection. These applications exhibit diverse characteristics:
 - Compute-intensive: TC, DIA, 4-cycle, K-Truss
 - Memory-bound and software-managed caching: BFS, PR, SpMV
 - High-intensity atomic operation: PR, K-Truss, K-Core
 - Load imbalance: DIA, 4-Cycle, Louvain
- UpDown efficiently supports all of these characteristics while maintaining high performance.

Fine-Grained Parallelism

UpDown achieves high overall compute efficiency by exploiting fine-grained thread invocations (10 to a few hundred instructions).

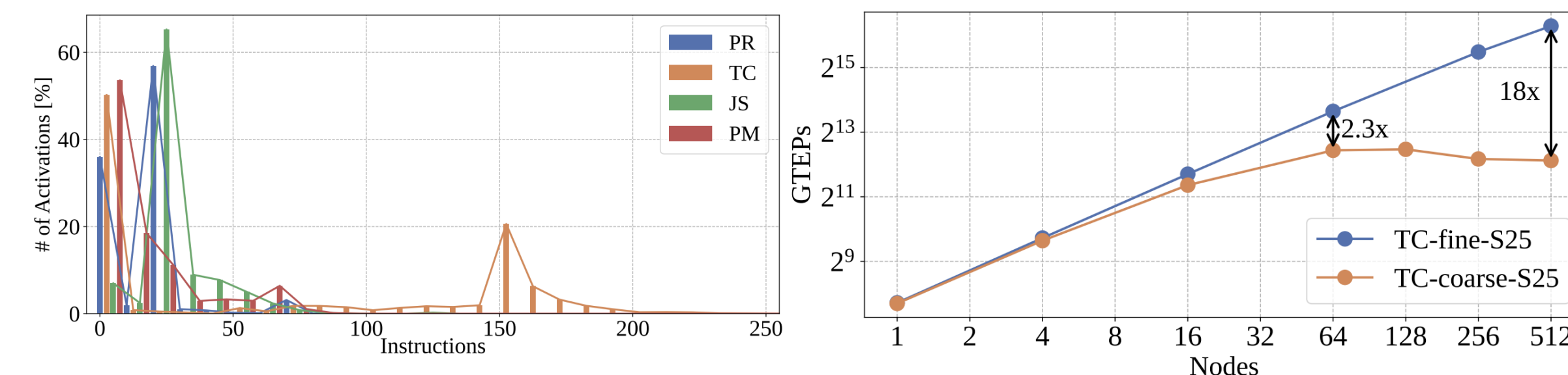


Figure 6. Histograms of instruction for PR, TC, JS, and PartialMatch (left); Fine-grained parallelism (vertex pairs) reduces the imbalance and produces improved scaling compared to coarse-grained parallelism (vertices).

Communication and Streaming

UpDown provides "network interface-less", low-overhead messaging, reducing the cost of sending a message globally in the machine to a single cycle. This large performance increase is the opportunity of co-design for fine-grained, irregular computations.

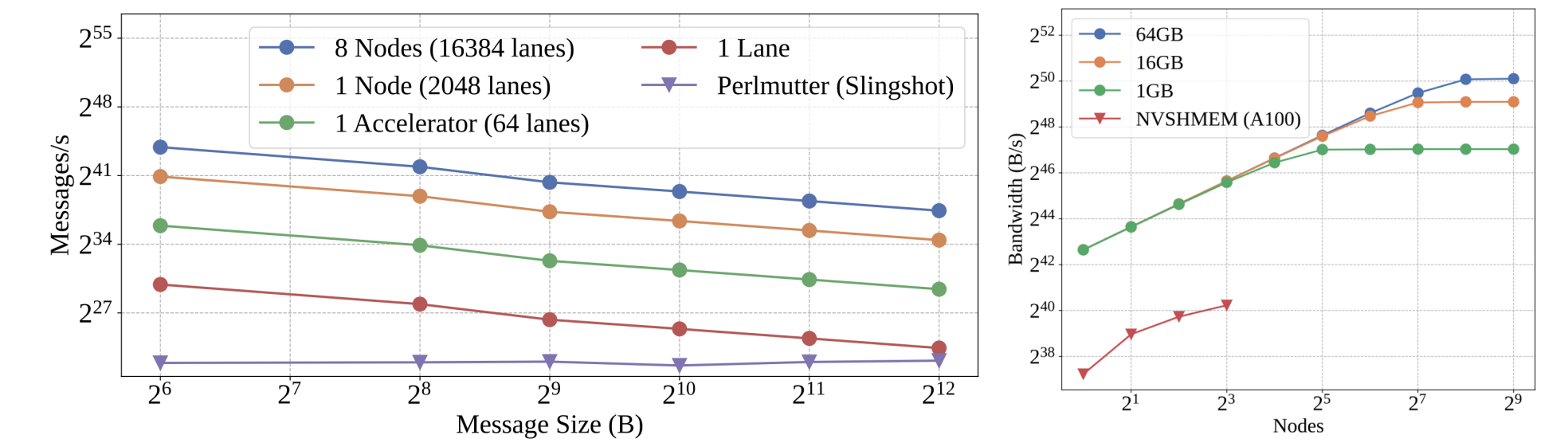


Figure 7. UpDown achieves 1 Giga-messages/sec/lane, 64 GigaM/s/accelerator, 2 TeraM/s/node (left). The software-based UpDown SHMEM libraries achieve 10 TB/s for 1 GB transfers on a single node (100x over A100) and scale efficiently with increasing numbers of nodes and data blocks (right).

Single-Node Performance

We compare UpDown GEM5 simulated performance to hand-tuned single-threaded programs run on conventional Intel Xeon CPU core. A single UpDown node delivers geomean speedups of **1,461–1,865x**, exploiting its 32 UpDown accelerators (2,048 simple lanes) with high efficiency.

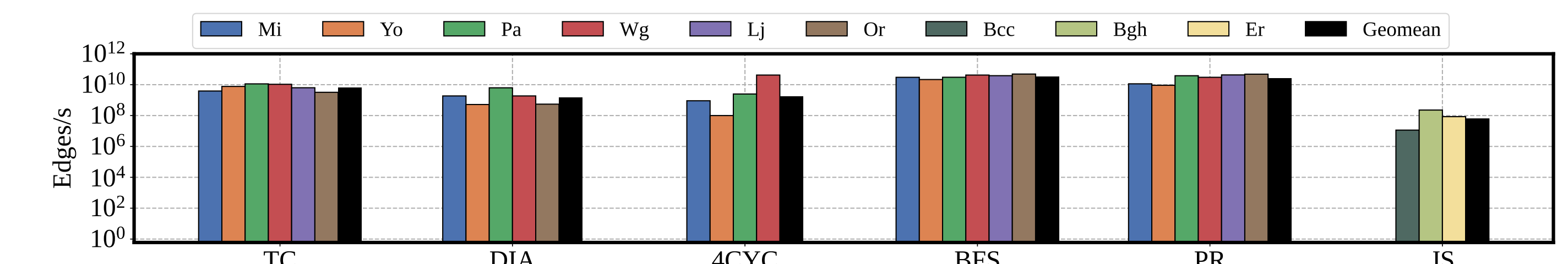


Figure 8. UpDown performance (Edges Per Second). Varied workloads and graphs.

System-Scale Applications

We evaluate the full UpDown system across 1 to 512 nodes (corresponding to 2,048 to 1,048,576 lanes), with additional data points at 16,384 nodes, and compare its performance with existing systems. PR and BFS achieve **9 TTEPS** on 512 nodes, BFS achieves **279 TTEPS** on 16k nodes, well above Graph 500 champion Fugaku's performance of 198 TTEPS using nearly ten times as many nodes (152,064 nodes)

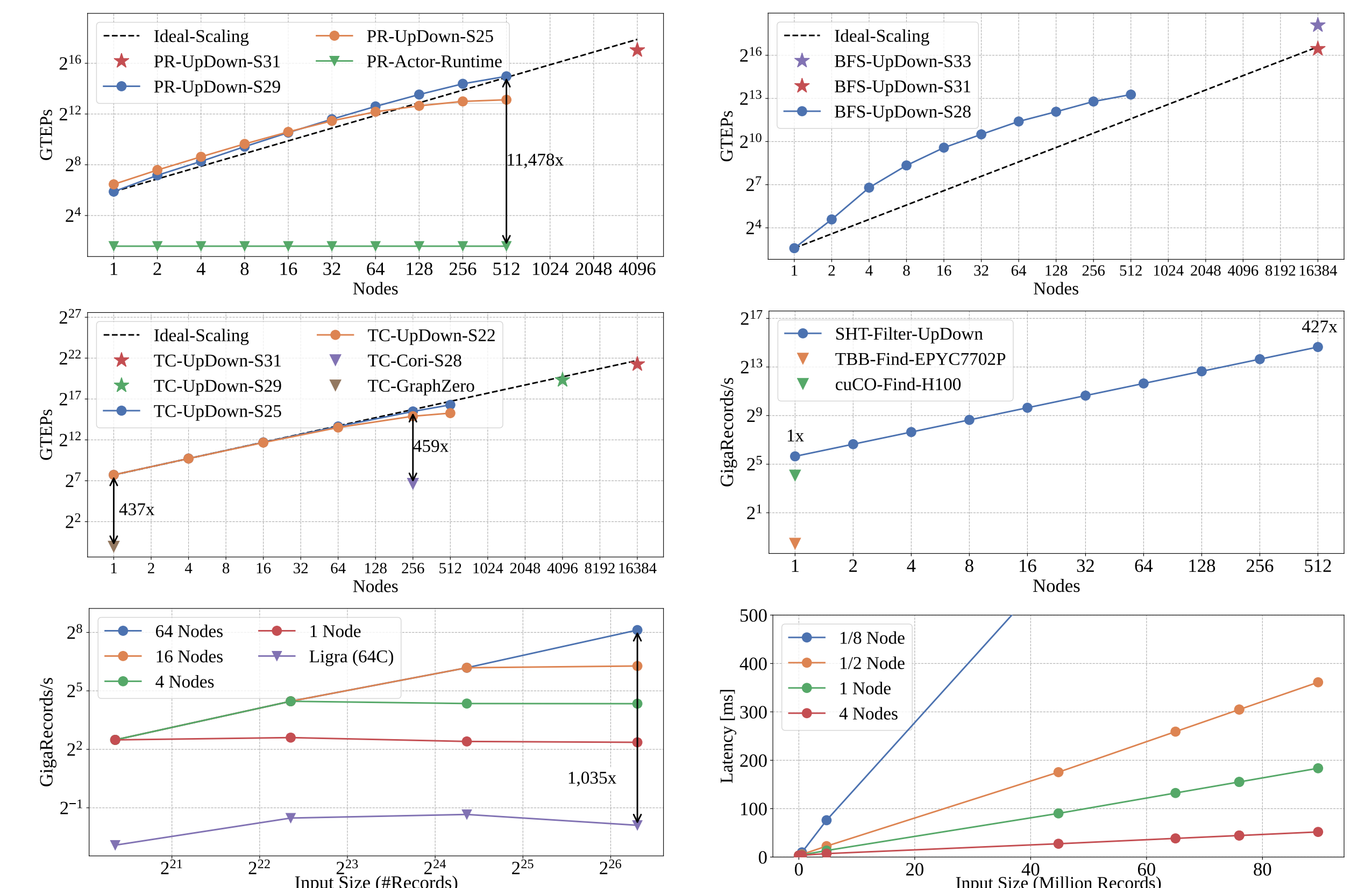


Figure 9. System-scale results for PR, BFS, TC, Scalable Hashing with Filtering, Graph Ingestion, and Partial Match.

Acknowledgements. Supported by ODNI/IARPA through AGILE under ARO contract W911NF22C0082, in part by NSF CNS-1907863, a Computation Innovation Fellows Award, and DOE contract DE-AC02-06CH11357.

Adapted from our TPDs paper: Chien, Andrew A., et al. "Updown: A supercomputer co-designed for scalable graph processing." IEEE Transactions on Parallel and Distributed Systems (2026).